

SAP ADT Python — Project Documentation

Version: 0.2.0 Created: 2026-04-02 Updated: 2026-04-02 Author: egoetz Purpose: ABAP Development Workbench via SAP ADT REST API + MCP Server for Cursor IDE

1. Overview

This project implements a Python client for the SAP ADT (ABAP Development Tools) REST API, the same HTTP endpoints that Eclipse ADT uses under `/sap/bc/adt/`. It includes an MCP (Model Context Protocol) server that allows Cursor IDE to function as a full ABAP development workbench.

Capabilities

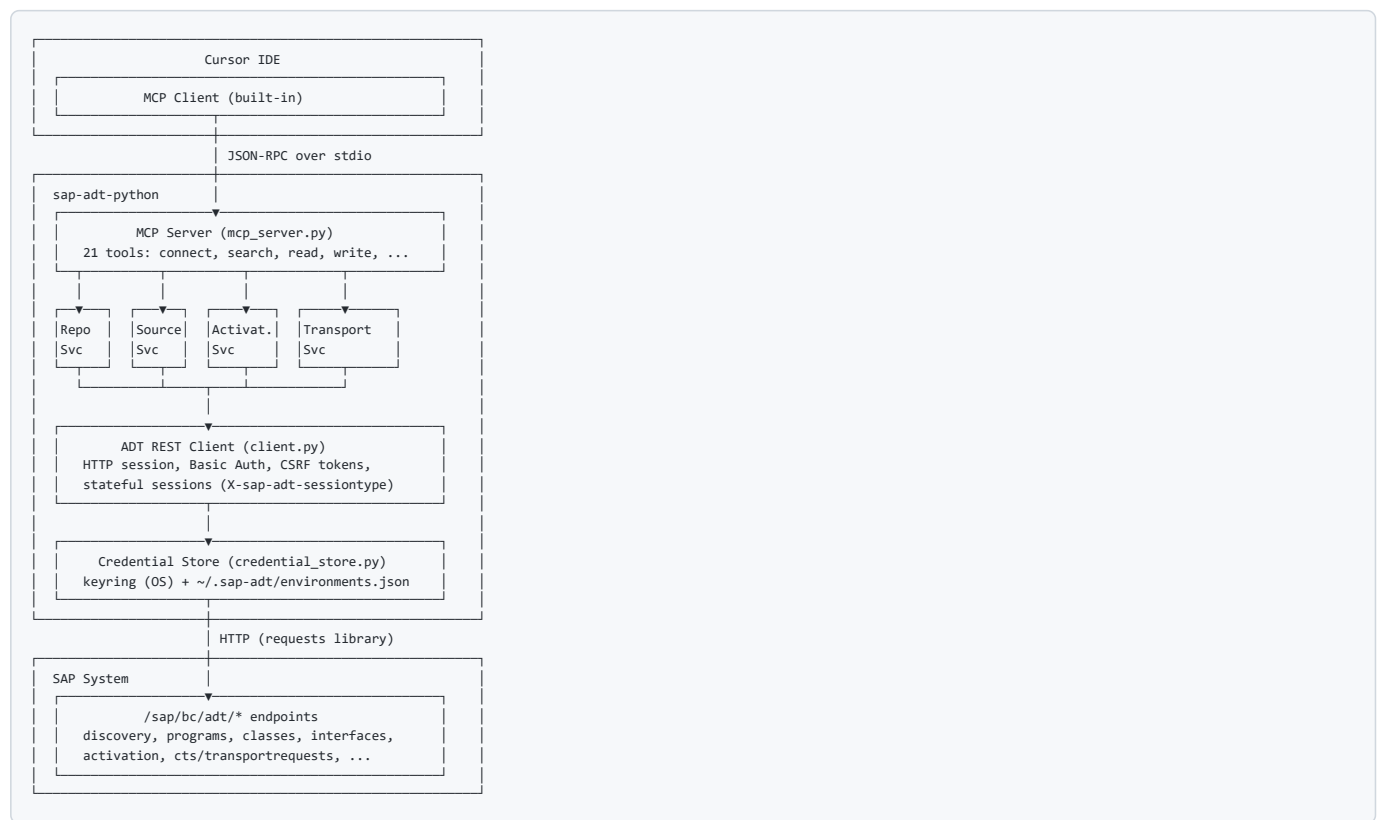
- **Connect** to SAP systems via HTTP Basic Auth + CSRF token management
- **Secure credential storage** in the OS credential manager (Windows Credential Manager, macOS Keychain, Linux Secret Service) via `keyring`
- **Multi-environment support** — save and switch between SAP systems (ED2, EQ2, PRD, etc.)
- **Search** the ABAP repository for programs, classes, interfaces, function groups, tables
- **Browse** packages (including namespace packages like `/TAX/SPED_MONITOR`) and view object metadata
- **Read/Write** ABAP source code with automatic lock management and stateful ADT sessions
- **Read from / save to local files** — export ABAP source to `.abap` files and upload from local files
- **Activate** (compile) objects with error reporting
- **Syntax check** without activating
- **Manage transports** — check locks, list/create/release transport requests
- **Create/Delete** ABAP objects (programs, classes, interfaces)

2. Target SAP Systems

Environments are stored securely via `sap_save_password` and loaded via `sap_connect(system_id="...")`.

Parameter	ED2 (Development)	EQ2 (Quality)
Host	<code>awsntwpstx01.engdb.infra</code>	<code>awsntwpstx01.engdb.infra</code>
Port	<code>8000</code>	<code>8008</code>
Protocol	HTTP	HTTP
Client	<code>500</code>	<code>500</code>
User	<code>egoetz</code>	<code>egoetz</code>
Language	<code>EN</code>	<code>EN</code>

3. Architecture



4. Project Structure

```
D:\ENGDB\sap-adt-python\
├── pyproject.toml           # Python project metadata and dependencies
├── requirements.txt         # Pip dependencies
├── .env.example             # Environment variable template
├── .gitignore              # Git ignore rules
├── README.md               # Quick-start guide
├── AGENTS.md               # Cursor agent instructions
├── .cursor/
│   ├── mcp.json            # MCP server configuration for Cursor
│   └── rules/
│       └── abap-dev.mdc    # Cursor rule for ABAP development workflow
├── docs/
│   ├── PROJECT_DOCUMENTATION.md # This file – project overview and architecture
│   └── MCP_COMMAND_REFERENCE.md # Complete MCP command reference with examples
├── src/
│   ├── sap_adt/
│   │   ├── __init__.py     # Package init
│   │   ├── client.py       # Core ADT HTTP client (auth, CSRF, stateful sessions)
│   │   ├── credential_store.py # Secure password storage via OS keyring
│   │   ├── models.py       # Data models (AbapObject, TransportRequest, etc.)
│   │   ├── exceptions.py   # Custom exception hierarchy
│   │   ├── repository.py   # Search, browse packages, object metadata
│   │   ├── source.py       # Read/write source code with lock management
│   │   ├── activation.py   # Activate objects, syntax check
│   │   ├── transport.py    # Transport request management (CTS)
│   │   └── mcp_server.py   # MCP server exposing 21 tools
│   ├── scripts/
│   │   ├── run_mcp.py      # MCP server launcher
│   │   └── test_connection.py # Connection test script
│   └── tests/
│       ├── __init__.py
│       ├── test_client.py  # Unit tests for client, models, URI resolution
│       └── test_repository_metadata.py # Unit tests for object metadata XML parsing
```

5. ADT REST API Endpoints Used

All endpoints are relative to the base URL (e.g. `http://awsntwpstx01.engdb.infra:8000`).

Authentication & Discovery

Method	Path	Accept Header	Purpose
GET	/sap/bc/adt/discovery	application/atomsvc+xml	Service discovery + CSRF token (via X-CSRF-Token: Fetch)

Repository

Method	Path	Accept Header	Purpose
GET	/sap/bc/adt/repository/informationssystem/search	application/xml	Quick search for ABAP objects
POST	/sap/bc/adt/repository/nodestructure	application/vnd.sap.as+xml	Package tree navigation
GET	/sap/bc/adt/programs/programs/{name}	application/vnd.sap.adt.programs.programs.v2+xml	Program metadata
GET	/sap/bc/adt/oo/classes/{name}	application/vnd.sap.adt.oo.classes.v4+xml	Class metadata
GET	/sap/bc/adt/oo/interfaces/{name}	application/vnd.sap.adt.oo.interfaces.v5+xml	Interface metadata

Source Code

Method	Path	Accept Header	Purpose
GET	/sap/bc/adt/programs/programs/{name}/source/main	text/plain	Read program source
GET	/sap/bc/adt/oo/classes/{name}/source/main	text/plain	Read class source
GET	/sap/bc/adt/oo/interfaces/{name}/source/main	text/plain	Read interface source
GET	/sap/bc/adt/functions/groups/{name}/source/main	text/plain	Read function group source
PUT	{source_uri}?lockHandle=...&corrNr=...	text/plain	Write source code (requires lock handle + stateful session)

Object Lifecycle

Method	Path	Accept Header	Purpose
POST	{object_uri}? _action=LOCK&accessMode=MODIFY	application/vnd.sap.as+xml	Lock object (stateful session)
POST	{object_uri}?_action=UNLOCK&lockHandle={h}	application/vnd.sap.as+xml	Unlock object (stateful session)
POST	/sap/bc/adt/activation	application/xml	Activate (compile) objects
POST	/sap/bc/adt/programs/programs	application/vnd.sap.adt.programs.programs.v2+xml	Create new program
POST	/sap/bc/adt/oo/classes	application/vnd.sap.adt.oo.classes.v2+xml	Create new class
POST	/sap/bc/adt/oo/interfaces	application/vnd.sap.adt.oo.interfaces.v2+xml	Create new interface
DELETE	{object_uri}	application/xml	Delete object

Transport Management (CTS)

Method	Path	Accept Header	Purpose
POST	/sap/bc/adt/cts/transportchecks	application/xml	Check if object needs transport
GET	/sap/bc/adt/cts/transportrequests	application/vnd.sap.adt.transportorganizer.v1+xml	List transport requests
POST	/sap/bc/adt/cts/transportrequests	application/xml	Create new transport request
POST	/sap/bc/adt/cts/transportrequests/{trkorr}/newreleasejobs	application/xml	Release transport

6. Authentication Flow

1. Initial request to /sap/bc/adt/discovery includes:

- Authorization: Basic <base64(user:password)>
- X-CSRF-Token: Fetch
- sap-client: 500
- sap-language: EN

2. Response contains:

- CSRF token in X-CSRF-Token header
- Session cookies (SAP_SESSIONID_*, sap-usercontext)

3. Subsequent requests include:

- The CSRF token in X-CSRF-Token header (for POST/PUT/DELETE)
- Session cookies (managed automatically by requests.Session)

4. On CSRF expiry (HTTP 403), the client automatically re-fetches the token and retries.

5. Password resolution order:

- Explicit password argument
- SAP_PASSWORD environment variable
- OS credential manager via keyring (if system_id provided)

7. Secure Credential Storage

Credentials are managed via the keyring library, which uses the native OS credential manager:

OS	Backend
Windows	Windows Credential Manager
macOS	Keychain
Linux	Secret Service (GNOME Keyring, KWallet)

Storage layout:

- **Passwords** → stored in OS keyring under service `sap-adt-mcp`, key `{system_id}:{user}`
- **Connection parameters** → stored in `~/.sap-adt/environments.json` (no secrets)

Workflow:

```
1. sap_save_password(system_id="ED2", host="...", user="egoetz", password="****")
2. sap_connect(system_id="ED2") → password retrieved from OS keyring
```

8. Write Source Code Workflow

The `sap_write_source` MCP tool performs the complete workflow:

1. **Transport check** — Determines if the object needs a transport request; auto-detects the recording transport
2. **Lock** — Acquires an edit lock on the object (`POST ?_action=LOCK`) with `X-sap-adt-sessiontype: stateful`
3. **Write** — Sends the new source code (`PUT .../source/main?lockHandle=...&corrNr=...`) with stateful session header
4. **Activate** — Compiles the object (`POST /activation`)
5. **Unlock** — Releases the lock (`POST ?_action=UNLOCK`) with stateful session header

If any step fails, the lock is released in the `finally` block.

Source input options:

- `source_code` — ABAP code as inline text
- `source_file` — path to a local `.abap` file (used when `source_code` is empty)

9. Object Metadata

The `sap_object_metadata` tool fetches detailed information about an ABAP object using vendor-specific `Accept` headers:

Object Type	Accept Media Type
Program (PROG)	<code>application/vnd.sap.adt.programs.programs.v2+xml</code>
Class (CLAS)	<code>application/vnd.sap.adt.oo.classes.v4+xml</code> (fallback: v2)
Interface (INTF)	<code>application/vnd.sap.adt.oo.interfaces.v5+xml</code> (fallback: v2)
Function Group (FUGR)	<code>application/vnd.sap.adt.functions.groups.v2+xml</code>

The metadata XML parser handles:

- Root-level attributes with `adtcore:*` namespace (e.g. `adtcore:name`, `adtcore:type`)
- Nested `atom:content` → `abapProgram` / `abapClass` elements
- Package from `adtcore:packageRef` child element
- Responsible user from `adtcore:responsible` attribute
- Source URI from `atom:link` elements (relative URIs resolved to absolute)

10. MCP Server Tools (21 tools)

Category	Tool	Description
Connection	<code>sap_connect</code>	Authenticate to SAP system (supports saved environments via <code>system_id</code>)
Connection	<code>sap_discovery</code>	List ADT services
Credentials	<code>sap_save_password</code>	Store credentials securely in OS credential manager
Credentials	<code>sap_delete_password</code>	Remove saved credentials
Credentials	<code>sap_list_environments</code>	List saved SAP environments (no passwords shown)
Repository	<code>sap_search</code>	Search ABAP objects (wildcards: <code>z*</code> , <code>zcl_*</code>)
Repository	<code>sap_browse_package</code>	List package contents (supports namespace packages)
Repository	<code>sap_object_metadata</code>	Get object details (name, type, package, responsible, source URI)
Source	<code>sap_read_source</code>	Read ABAP source code (optional: save to local file)
Source	<code>sap_write_source</code>	Write source + auto-lock + transport + activate (from text or file)
Source	<code>sap_activate</code>	Compile/activate an object
Source	<code>sap_syntax_check</code>	Syntax check without activating
Transport	<code>sap_check_lock</code>	Check if object is locked and by whom
Transport	<code>sap_list_transports</code>	List open (modifiable) transport requests
Transport	<code>sap_create_transport</code>	Create new workbench transport request
Transport	<code>sap_release_transport</code>	Release a transport request
Transport	<code>sap_transport_check</code>	Check if object needs transport, list available transports
Create	<code>sap_create_program</code>	Create new ABAP program (report)
Create	<code>sap_create_class</code>	Create new ABAP global class
Create	<code>sap_create_interface</code>	Create new ABAP interface
Delete	<code>sap_delete_object</code>	Delete an ABAP object (requires <code>confirm='DELETE'</code>)

For complete parameter details, examples, and SAP prerequisites, see [MCP_COMMAND_REFERENCE.md](#).

11. Key Technical Details

Stateful Sessions

SAP ADT requires `X-sap-adt-sessiontype: stateful` header on lock, write, and unlock operations. This ensures the lock handle remains valid across multiple HTTP requests within the same session.

Vendor-Specific Accept Headers

Many ADT endpoints reject `application/xml` and require vendor-specific media types (e.g. `application/vnd.sap.adt.programs.programs.v2+xml`). The client automatically selects the correct `Accept` header based on the object URI.

Namespace Package Handling

Namespace packages like `/TAX/SPED_MONITOR` require percent-encoded slashes in URIs: `%2ftax%2fsped_monitor` . The `browse_package` method uses `POST` to `/sap/bc/adt/repository/nodestructure` with query parameters (`parent_name` , `parent_tech_name` , `parent_type` , `withShortDescriptions`).

Transport Request Parameters

The transport request number (`corrNr`) and lock handle (`lockHandle`) are sent as **query parameters** on the `PUT` request, not as HTTP headers.

12. Setup Instructions

Prerequisites

- Python 3.10+
- Network access to SAP system (e.g. `awsntwpstx01.engdb.infra:8000`)
- SAP user with ADT authorization (see [MCP_COMMAND_REFERENCE.md § 6](#))

Installation

```
cd D:\ENGDB\sap-adt-python
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
```

First-time Credential Setup

```
sap_save_password(
    system_id="ED2",
    host="awsntwpstx01.engdb.infra",
    port=8000,
    client="500",
    user="egoetz",
    password="****"
)
```

Connection Test

```
python scripts\test_connection.py
```

Cursor MCP Configuration

The workspace includes `.cursor/mcp.json` with server key `sap-adt` . In Cursor the server is shown as `user-sap-adt` .

You can copy the same block to `~/.cursor/mcp.json` (global) or adjust paths:

```
{
  "mcpServers": {
    "sap-adt": {
      "command": "python",
      "args": ["D:\\ENGDB\\sap-adt-python\\scripts\\run_mcp.py"],
      "env": {
        "SAP_HOST": "awsntwpstx01.engdb.infra",
        "SAP_PORT": "8000",
        "SAP_CLIENT": "500",
        "SAP_USER": "egoetz",
        "SAP_LANGUAGE": "EN"
      }
    }
  }
}
```

With `sap_save_password` , `SAP_PASSWORD` is not needed in the config — the password is retrieved from the OS credential manager.

13. Dependencies

Package	Version	Purpose
requests	>=2.31	HTTP client for ADT REST API
mcp	>=1.0	Model Context Protocol SDK (FastMCP)
lxml	>=5.0	XML parsing for ADT responses
keyring	>=25.0	Secure credential storage in OS keyring

14. Technology References

- SAP ADT REST API: Same HTTP endpoints used by Eclipse ADT plugin (`/sap/bc/adt/*`)
- SAP ADT Configuration Guide: [Configuring the ABAP Back-end for ADT](#)
- SAP Development Tools: <https://tools.hana.ondemand.com/>
- MCP Protocol: Model Context Protocol for AI tool integration
- keyring library: <https://pypi.org/project/keyring/>
- erpl-adt: Reference CLI implementation (<https://github.com/DataZooDE/erpl-adt>)
- abap-adt-py: Reference Python library (<https://pypi.org/project/abap-adt-py/>)

15. Related Documentation

Document	Description
README.md	Quick-start guide
AGENTS.md	Cursor agent instructions
MCP_COMMAND_REFERENCE.md	Complete command reference with parameters, examples, and SAP prerequisites
.cursor/rules/abap-dev.mdc	Cursor rule for ABAP development workflow